



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



PREVIO N° 01

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

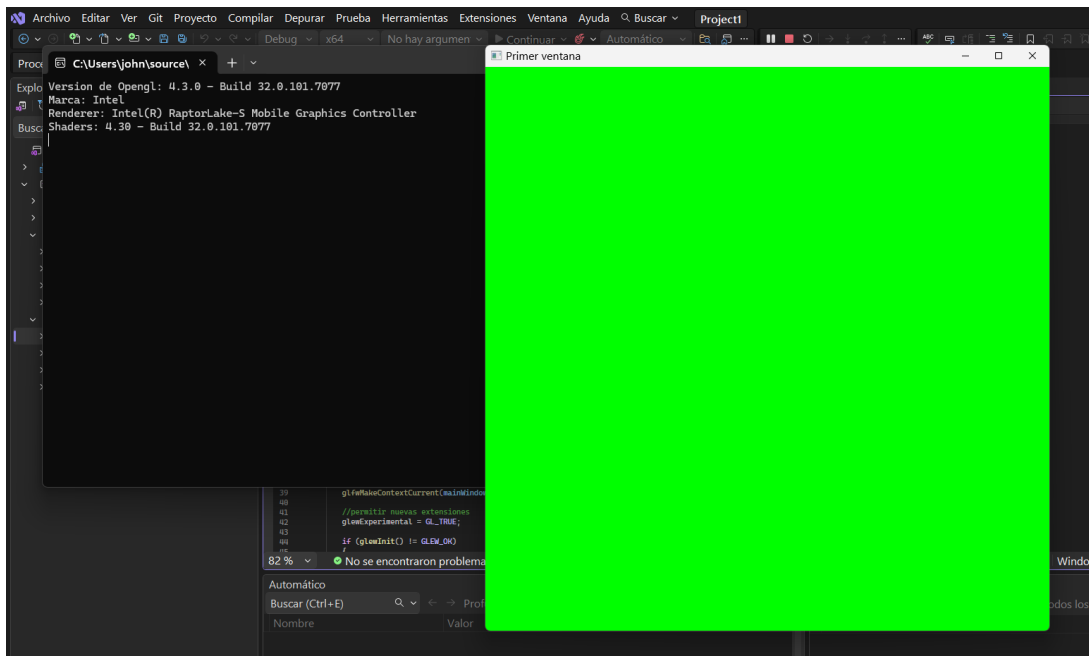
GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 12/02/2026

CALIFICACIÓN: _____

Captura de pantalla como la del manual de configuración en la cual se muestra la ventana de fondo verde y la información de la consola con los datos de Hardware de su equipo de cómputo



Cuestionario Previo 1:

¿Qué es un VAO?

R= Un Vertex Array Object (VAO) es un objeto de OpenGL que almacena el estado de la configuración de los atributos de vértices, incluyendo que VBOs están ligados y cómo se interpretan sus datos. Permite reutilizar esa configuración sin volver a especificarla cada vez que se dibuja (Learn OpenGL - Hello Triangle, n.d)

¿Qué es un VBO?

R= Un Vertex Buffer Object es un buffer de memoria en la GPU que almacena datos de vértices (posiciones, colores, normales, etc.) para ser usados por la tubería de renderizado. Al estar en la memoria de la tarjeta gráfica, mejora el rendimiento frente a enviar datos cada vez desde la CPU (Wikipedia Vertex Buffer Object)

d) ¿Qué parámetros recibe el comando: `glVertexAttribPointer`?

R= El comando `glVertexAttribPointer` recibe seis parámetros que definen como se leerá un array de atributos de vértices genéricos:

- 1) index (`GLuint`): Especifica el índice del atributo de vértice que se va a modificar
- 2) size (`GLint`): Número de componentes por atributo (debe ser 1, 2, 3, 4 o `GL_BGRA`)
- 3) type (`GLenum`): Tipo de dato de cada componente (ej. `GL_FLOAT`, `GL_UNSIGNED_BYTE`, `GL_INT`)
- 4) normalized (`GLboolean`): Indica si los valores de punto fijo deben normalizarse al rango $[0, 1]$ ó $[-1, 1]$ al ser convertidos a punto flotante
- 5) stride (`GLsizei`): Desplazamiento en bytes entre atributos de vértices consecutivos. Si es 0, se asume que los datos están empaquetados juntos
- 6) pointer (`const_void*`): Especifica el desplazamiento (offset) en bytes del primer componente dentro del buffer actualmente vinculado a `GL_ARRAY_BUFFER`

(`GLAPI / glVertexAttribPointer` - OpenGL Wiki, nd)
(Khronos Group `glVertexAttribPointer`)

¿Qué información maneja Fragment Shader?

R = El Fragment Shader (sombreador de fragmentos) maneja la información necesaria para generar el color final de cada fragmento generada por la rasterización, procesa:

- A) Entradas del sistema: `gl_FragCoord` (posición en ventana), `gl_FrontFacing` (orientación del polígono), `gl_PointCoord` (coordenadas dentro de un punto).
- B) Variables de entrada interpoladas: Datos provenientes del vertex shader (como normales, coordenadas de textura) que son automáticamente interpoladas a través de la superficie del polígono.
- C) Texturas y muestreadores: Para aplicar mapeo de texturas.
- D) Salidas: El color final (a cero o más buffers de color) y opcionalmente un valor de profundidad personalizado (`gl_FragDepth`).

(Fragmented Shader - OpenGL wiki, no)

¿Qué parámetros recibe el comando `glDrawArrays`?

R = El comando `glDrawArrays` recibe tres parámetros para renderizar primitivas a partir de datos de array

- 1) `mode (GLenum)`: Especifica el tipo de primitiva a renderizar (ej. `GL_TRIANGLES`, `GL_LINES`, `GL_POINTS`, `GL_TRIANGLE_STRIP`).
 - 2) `first (GLint)`: Índice inicial en los arrays habilitados desde el cual comienza a leer datos.
 - 3) `count (GLsizei)`: Número de índices (vértices) a ser procesados.
- (SteveuHims `glDrawArrays` Function Microsoft Learn)

Conclusión:

El cuestionario previo me ayudó a entender mejor los elementos básicos que intervienen en la computación gráfica, especialmente en el uso de OpenGL. Al revisar conceptos como VAO y VBO pude comprender los elementos básicos de como se organizan los datos para dibujar objetos en pantalla.

Otro elemento importante fue entender los llamados Shaders (Fragment y Vertex) respecto a como se procesan los vértices y como se define el color final de las imágenes.

Este cuestionario me permitió tener una visión más clara del flujo de trabajo gráfico, lo cual me resultara útil al momento de realizar la práctica.

Bibliografía

- LearnOpenGL. (s. f.). Hello Triangle. Recuperado de <https://learnopengl.com/Getting-started/Hello-Triangle>
- Wikipedia. (s. f.). Vertex buffer object. Recuperado de https://en.wikipedia.org/wiki/Vertex_buffer_object
- Khronos Group. (s.f.-b). `glVertexAttribPointer`. OpenGL 4 Reference Pages. Recuperado el 11 de febrero de 2026, de <https://registry.khronos.org/OpenGL-Refpages/gl4/html/glVertexAttribPointer.xhtml>
- OpenGL Wiki. (s. f.). `glVertexAttribPointer`. Recuperado de <https://wikis.khronos.org/opengl/GLVertexAttribPointer> Shader Tutorial. (s.f.). Shader Basics - Vertex Shader. GPU Shader Tutorial. Recuperado el 11 de febrero de 2026, de <https://shader-tutorial.dev/basics/vertex-shader/>
- Khronos Group. (s.f.-d). Fragment Shader. OpenGL Wiki. Recuperado el 11 de febrero de 2026, de https://wikis.khronos.org/opengl/Fragment_Shader
- Microsoft Learn. (s. f.). `glDrawArrays`. Recuperado de <https://learn.microsoft.com/en-us/windows/win32/opengl/glDrawArrays>